

CHEM 273 Discussion 2

Usman Jamshed

Housekeeping

- Submit exercises and homeworks in the .ipynb format to Gradescope
- Please use the homework file or clearly label code (i.e. Question 1, Question 2, etc.)
- You can submit regrades if you believe your grade is incorrect.
- Generally it's a good idea to put name, class, date in the top of your assignments for your own reference.
- Lecture Exercise 1 solutions and solution video are available on bCourses.
- Lecture Exercise 2 is due on Monday at 11:59 PM PST.

Lecture 1 Assignment Solution Walkthrough

Lecture 1 Assignment Feedback

- Comments vs Markdown in notebooks.
- Code running does not mean its correct.

Notebook Demo

<https://www.markdownguide.org/cheat-sheet/>

Style

Which one of these functions do you prefer? Why?

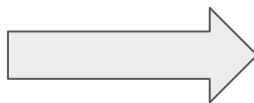
```
def read(x, y):  
    b = pd.read_excel(x)  
    return b
```

```
# function to read in an excel  
file using the pandas library  
  
def read_xlsx_pd(filename):  
    df = pd.read_excel(filename)  
    return df
```

Return Statements

[]:

```
def read_1(filename):  
    df = pd.read_csv(filename)  
    return
```



nothing

[]:

```
def read_2(filename):  
    df = pd.read_csv(filename)  
    return df
```



The dataframe

[]:

```
def read_3(filename):  
    df = pd.read_csv(filename)  
    return df.head()
```



The first five lines of the
dataframe

Common errors

Function names should be descriptive and **unique**

Functions only need to be defined once, but can be called multiple times with different inputs

Organize your code well and include comments when needed

For loops

- A for loop lets you repeat code for a series of values
- Syntax

```
for i in range(N):  
    # code
```

- range(N) gives the numbers from 0 to (N - 1)
- i becomes each of these numbers for each turn through the loop
- Try out this example:

```
for i in range(5):  
    print("i is now:", i)
```

```
i is now: 0  
i is now: 1  
i is now: 2  
i is now: 3  
i is now: 4
```


For loops

Without coding this, what will the output be:

```
for i in range(4):  
    print(i*2)
```

i	Output (i * 2)
0	0
1	2
2	4
3	6

Range

- The range() function returns a sequence of numbers starting from 0 (by default) and increasing by increments of 1 (by default) and stops before a specific number
- Syntax

```
range(start, stop, step)
```

- Start: optional, number of where to start, default is 0
- Stop: required, number of where to stop (sequences will stop before this number is reached)
- Step: optional, number of how much to increment, default is 1

If you include step, you must include start

Range practice

```
range(start, stop, step)
```

```
range(0, 100, 10)
```

0, 10, 20, 30, 40, 50, 60, 70, 80, 90

```
range(5, 12)
```

5, 6, 7, 8, 9, 10, 11

```
range(-2, 6, 2)
```

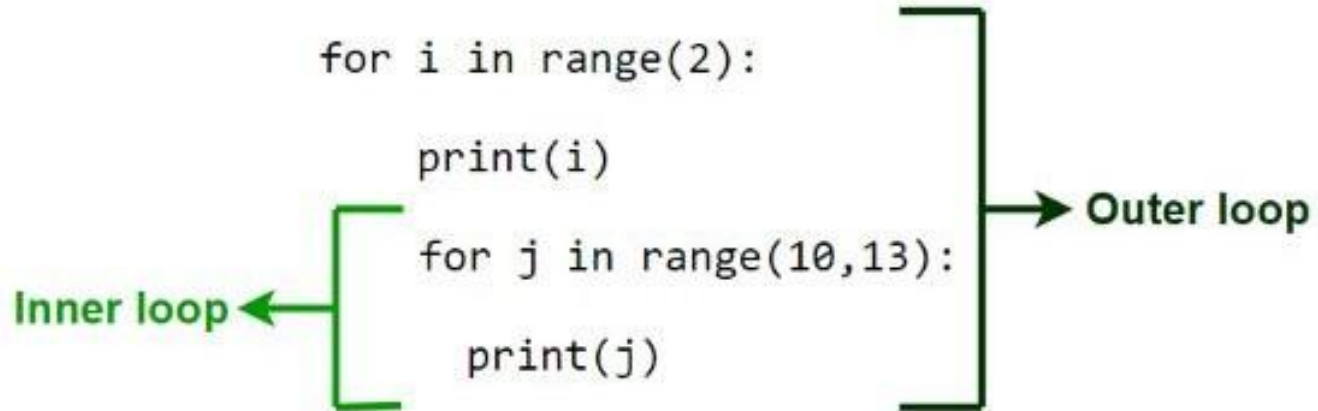
-2, 0, 2, 4

```
range(10, 0, -2)
```

10, 8, 6, 4, 2

Nested for loops

Nested for loops are loops inside of a loop



Nested for loops

Example:

```
# nested for loop  
for i in range(3):  
    print("i is:", i)  
    for j in range(3):  
        print("j is:", j)
```

i is: 0
j is: 0
j is: 1
j is: 2
i is: 1
j is: 0
j is: 1
j is: 2
i is: 2
j is: 0
j is: 1
j is: 2

np.array

NumPy is often used to work with arrays

When arrays only have one dimension (like a list, they are referred to as 1D)

Example: Make a 1D Numpy Array

[26]:

```
import numpy as np
```

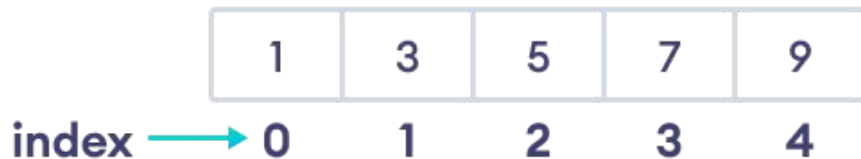
[30]:

```
arr = np.array([5, 2, 6, 8, 1])  
print(arr)
```

```
[5 2 6 8 1]
```

np.array

Indexes start at 0



To index in a 1D array, simply use the name of the array, hard brackets and the index that you want:

```
arr = np.array([5, 2, 6, 8, 1])
```

```
arr[3]
```

-> 8

np.array

2D arrays are similar to tables or matrices

Example:

```
new_arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(new_arr)
```

```
[[1 2 3]  
 [4 5 6]]
```


np.array

Indexing in a 2D array:

```
arr[row, column]
```

Practice indexing:

`new_arr[0, 1]`

First row, second column =

2 `new_arr[1, 2]`

Second row, third column =

6 `new_arr[0, 0]`

First row, first column =

1 `new_arr[2, 1]`

Doesn't exist → third row!

[1 2 3]

[4 5 6]

np.tile

- Np.tile repeats an array to make a bigger array with a repeated pattern
- Syntax:

```
np.tile(arr, repetitions)
```

- Arr is the array you want to repeat and repetitions is the number of repetitions

Example

```
arr = [1, 0]  
np.tile(arr, 4)
```

[46]:

```
array([1, 0, 1, 0, 1, 0, 1, 0])
```

np.tile

- We can also use np.tile on a 2D array
- Syntax:

```
np.tile(array, (rows_repeated, columns_repeated))
```

Example

```
b = np.array([[1, 2, 3]])  
tiled = np.tile(b, (3, 2))  
print(tiled)
```

```
[[1 2 3 1 2 3]  
 [1 2 3 1 2 3]  
 [1 2 3 1 2 3]]
```

np.random.uniform

- np.random.uniform generates random decimal numbers
- Syntax: `np.random.uniform(low, high, size)`
-
- numbers will be generated between low and high, and size determines how many random numbers
- Example:

```
np.random.uniform(0, 1, 5)
```

```
[43]:
```

```
array([0.54195765, 0.10066753, 0.94523536, 0.30064311, 0.40679133])
```